

Lista 10 – Algoritmos e Estruturas de Dados

Tópico: Pilhas e Filas em Alocação Encadeada

28 de maio de 2025

PARA CADA QUESTÃO, ESCREVA O ALGORITMO EM PSEUDOCÓDIGO E JUSTIFIQUE AS DECISÕES DE PROJETO. ÍNDIQUE O TEMPO DE EXECUÇÃO DE CADA SOLUÇÃO.

1. **Operações básicas de pilha encadeada.** Considere a pilha encadeada ilustrada a seguir.



- Mostre, desenhando os nós e ponteiros, o estado da pilha após executar `Push(topo, 99)` e depois `Push(topo, 3)`.
 - A partir do estado obtido em (a), mostre o estado após dois `Pop`. Qual valor foi removido em cada chamada?
 - Por que o `Pop` deve guardar o nó removido em uma variável auxiliar antes de avançar o ponteiro `topo`? O que aconteceria se o nó fosse liberado da memória imediatamente, antes de atualizar `topo`?
2. **Fila encadeada com ponteiros início e fim.** Considere a fila encadeada abaixo, onde a remoção ocorre em `início` e a inserção em `fim`.



- Insira os valores 7 e 11 na fila e desenhe o estado resultante.
 - Realize dois `Dequeue` e desenhe o estado resultante.
 - Descreva com precisão o que deve ocorrer com os ponteiros `início` e `fim` quando a última remoção esvazia a fila. O que acontece se apenas `início` for zerado, mas `fim` não?
3. **Inversão de fila com pilha auxiliar.** Dada uma fila encadeada com n elementos, projete um algoritmo que inverta a ordem dos elementos usando apenas uma pilha encadeada auxiliar (sem vetores ou outras estruturas).
- Escreva o pseudocódigo e trace sua execução para a fila $\langle 1, 2, 3, 4 \rangle$ (onde 1 está em `início`).
 - Análise a complexidade de tempo e espaço.
 - É possível inverter a fila *in-place*, sem nenhuma estrutura auxiliar, usando apenas manipulação de ponteiros? Justifique.
4. **Pilha com mínimo em $O(1)$.** Projete uma pilha encadeada que, além das operações usuais `Push` e `Pop`, suporte a operação `Min`, que retorna o menor elemento presente na pilha em tempo $O(1)$ — sem percorrer a lista.

- (a) Descreva a estrutura de dados (quais campos cada nó deve conter).
 - (b) Escreva os pseudocódigos de Push, Pop e Min, todos em $O(1)$.
 - (c) Trace a execução para a sequência de operações:
Push(5), Push(3), Push(7), Push(1), Pop, Min.
Mostre o estado da pilha e o valor mínimo registrado em cada passo.
 - (d) Qual é o custo extra de memória em relação a uma pilha encadeada comum? Expresse em $O(\cdot)$.
5. **Balanceamento de delimitadores.** Uma expressão pode conter três tipos de delimitadores: parênteses $()$, colchetes $[]$ e chaves $\{\}$. Um algoritmo baseado em pilha verifica se os delimitadores estão corretamente aninhados.
- (a) Escreva o algoritmo usando uma pilha encadeada. Para cada caractere da entrada: se for abridor, empilhe; se for fechador, desempilhe e verifique o par.
 - (b) Analise a complexidade de tempo e espaço em função do comprimento n da expressão.
 - (c) Aplique o algoritmo às cadeias abaixo e indique se são válidas. Para as inválidas, identifique a posição e o tipo do erro.
 - $(\{[]\})$
 - $(\{[]\})]$
 - $(\{[]\}[(\{[]\})\{\}])$
6. **Fila com duas pilhas encadeadas.** É possível simular uma fila usando duas pilhas, chamadas P_1 (entrada) e P_2 (saída). A inserção sempre empilha em P_1 . A remoção retira o topo de P_2 ; se P_2 estiver vazia, todos os elementos de P_1 são transferidos para P_2 antes da remoção.
- (a) Escreva os pseudocódigos de Enqueue e Dequeue usando essa ideia.
 - (b) Mostre que o custo *amortizado* de Dequeue é $O(1)$, mesmo que uma chamada individual possa custar $O(n)$. *Sugestão:* use o método do potencial, definindo Φ = número de elementos em P_1 .
 - (c) A transferência de P_1 para P_2 preserva a propriedade FIFO? Justifique com um exemplo.
7. **Radix Sort e estabilidade.** O Radix Sort processa dígitos do menos significativo para o mais significativo e depende de que a distribuição nas filas seja *estável*.
- (a) Explique, com um contra-exemplo concreto, o que ocorreria se as filas fossem substituídas por pilhas no passo de coleta.
 - (b) Trace a execução do Radix Sort para a sequência 481, 329, 143, 612, 937, 480, 256, mostrando o conteúdo das 10 filas e a sequência coletada após cada passagem.
 - (c) Qual é a complexidade do Radix Sort quando aplicado a n strings de comprimento fixo ℓ sobre um alfabeto de tamanho σ ? Compare com Merge Sort para o mesmo problema.